AMS DB MEETUP · AMSTERDAM · 11 NOV 2026

Building a Postgres-compatible database with autonomous agents

PostgreSQL-compatible TiDB Cloud Starter: what it is, how compatible it is, and the code factory of agents that builds it.

Ian Zhai
TiDB / PingCAP

`AI agent wrangler. I ask for evidence when the agents say “done.”`

THE PRODUCT PUBLIC PREVIEW

Postgres on TiDB Cloud.
Start for free.

PostgreSQL-compatible
TiDB Cloud Starter

- A standard Postgres connection string: psql, drivers, ORMs, GUI tools
- Serverless: nothing to size, nothing to manage

Start for free →

tidbcloud.com/free-trial

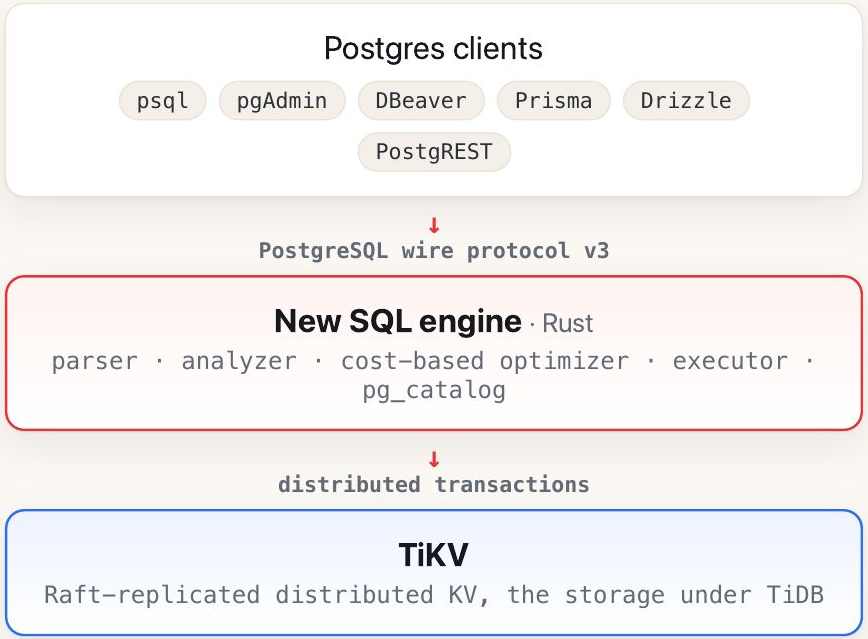
```
$ psql "postgresql://…@gateway01.us-east-1.prod.aws.pg.tidbcloud.com:5432/postgres?sslmode=verify-full"

postgres=> CREATE TABLE notes (
             id     bigserial PRIMARY KEY,
             owner  uuid NOT NULL,
             body   jsonb );
CREATE TABLE
postgres=> ALTER TABLE notes ENABLE ROW LEVEL SECURITY;
ALTER TABLE
```

UNDER THE HOOD

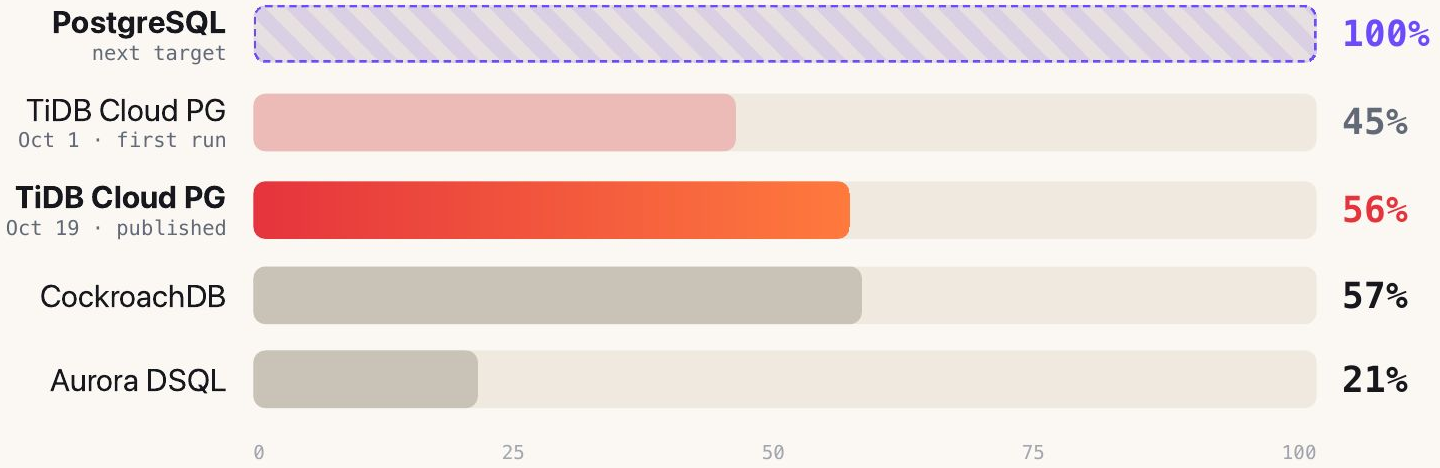
A new engine, not a fork.

- Every layer above storage is new code, written in Rust
- Speaks the Postgres wire protocol and SQL dialect
- Stores data in TiKV: Raft-replicated, distributed transactions
- Read Committed by default, Repeatable Read supported



COMPATIBILITY · MOVING FAST

+11 points in 18 days.



Next target: real Postgres.

PostgreSQL Compatibility Index, pgscorecard.com (open source)

measured by Daniël van Eeden

WHERE WE FOCUS

Supabase + TiDB Cloud PG

The most demanding real-world stack on Postgres. On every request:

- switch role, inject JWT claims with `set_config`
- enforce row-level security on every table
- PostgREST introspects `pg_catalog`, reloads on NOTIFY
- Auth and Storage run their own migrations

✓	PostgREST	unmodified
✓	Auth: sign-up, login, refresh	
✓	Row-level security across users & projects	
✓	Storage: private files, signed URLs	
✓	Edge Functions → REST / Storage	
✓	Realtime	
…	Supabase Studio	in progress
…	Postgres extensions	in progress

Stock Supabase images, no patches.

HOW IT WAS BUILT

~6 months. 2 humans. Countless agents.

~1M

lines of Rust in the engine

966K, no blanks or comments

15,129

commits, engine + cloud

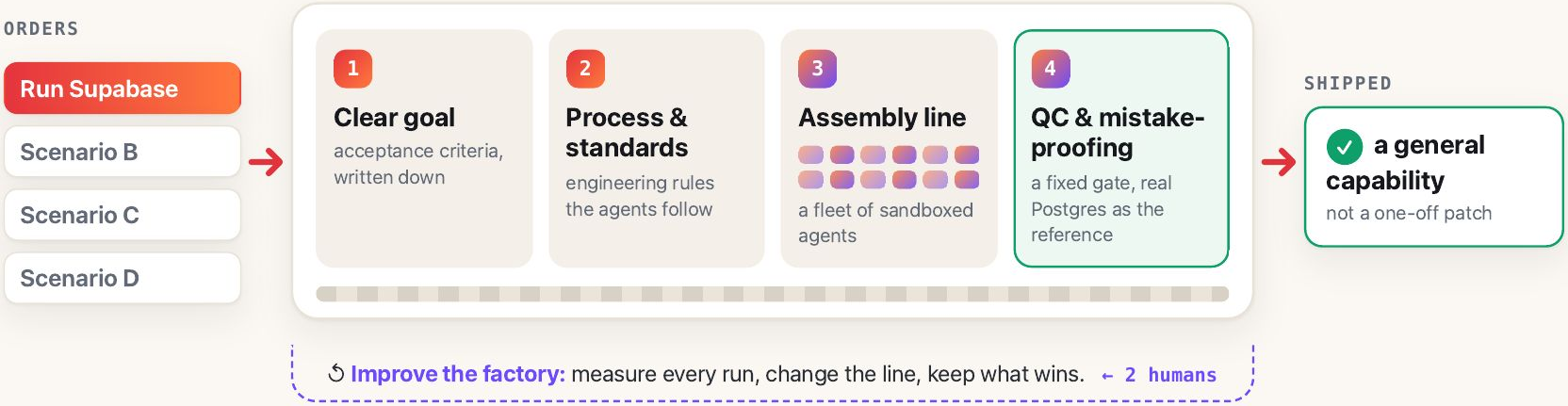
13,159 PRs merged · 48 repos

10,734

Rust tests

+ 977 SQL test files

Imagine a software factory.
That's how TiDB Cloud PG is built.



Nobody can read every line. So our job is the factory, not the code.

PART 2 · QUALITY CONTROL

~70,000 lines in 4 days. How do you control quality?

QUALITY CONTROL

The obvious answers don’t scale.

✗

Review every line?

Nobody can read 70,000 lines.

✗

More tests?

Necessary. Not sufficient.

✗

A better spec?

Waterfall with a new name.

> “Cease dependence on inspection to achieve quality … by building quality into the product in the first place.”
>
> — W. Edwards Deming, 1982

THE FACTORY BLUEPRINT

Three layers. Quality is built into each.

Harness
how work flows

- Clear goals
- Process & standards
- Issue grouping & triage
- Automated pipeline
- Quality gates

Infrastructure
what it runs on

- Fast CI/CD, or it backfires
- Merge / revert queue

Codebase
what agents read

- Agent-friendly: less is more

LAYER 1 · HARNESS MISTAKE-PROOFING

Harness | Infrastructure | Codebase

A gate said “pass”. It was grading its own homework.

THE GATE THAT LIED

A performance gate: **DDL overhead ≤ 2×**. It passed. But its baseline already included the change it was judging. Result: void.

STOP THE LINE

Toyota lets any worker pull a cord and stop the line. Our version: **fail 3 times → stop, step back, write down why.**

Gates need gates.

Slow tests make agents sloppy.

- Tests over ~20 minutes: agents fill the wait with busywork, and lose the thread
- So we're splitting our slowest test suite from **~50 minutes down to ~21**
- Agents multiply CI load by 10×+, so the full suite runs after merge, smoke tests before
- The opposite failure: testing every tiny edit. Too slow, too safe.
- **Revert queue = product recall.** If something breaks after it ships, it rolls back automatically, no waiting for a human. Early on we recalled ~100 features and re-delivered them

---

Slow tests aren't a productivity problem.
They're a quality problem.

Agents read everything. Including what's stale.

Three production configs, **two of them stale**. An agent read the wrong one.

The agent rules file kept growing. We **split it into small files**, named to be found, read on demand.

Agents add things nobody remembers. **Delete the old path** when you replace it.

For agents, every extra file is a distraction.

THE TWIST

The factory runs. The gates are green. It can still build the wrong thing.

THE BLUEPRINT

What you wrote down

goals, specs, rules, acceptance criteria

?

Details nobody specified

every one becomes an agent's guess

THE PRODUCT

What actually gets built

the codebase, real Postgres behavior, real users

Blueprint → product: thousands of small decisions nobody saw being made.

ONE 4-DAY TASK

We wrote the spec first: 13 requirements. Then the territory talked back.

35

decisions made along the way

84

findings nobody planned for

10+

reversed mid-flight

Open the decision log ↗

At this scale, code quality is decision quality.

METHOD 1 · BEFORE YOU START

Write the order as numbers. Then let the agent ask you.

NUMBERS, NOT WISHES

R1 Hot OLTP statements: **zero** schema-storage round trips.
R3 DDL latency **close to TiDB**, no worse than CockroachDB.
13 requirements, each one checkable.

QUESTIONS ONLY YOU CAN ANSWER

Don’t know how it works? **Ask the agent to explain.**
Haven’t decided? **The agent asks you.**
Q1 Must REVOKE / GRANT take effect immediately?
→ yes → design changed.

Answering “A or B?” beats a blank page.

METHOD 2 · WHILE IT RUNS

Two lines, one log.

Build line	agents write the code, PR by PR
Verify line	runs in parallel: a TLA+ model, reviews, tests
The log	every finding lands here first, then changes the build line `OVERTURNED` `REFINED` `ADJUSTED` `PENDING`

`F-05 · Overturned` Per-round, per-server acks don't scale → switched to leases.

Don't wait for the end to find out.

METHOD 3 · BEFORE YOU MERGE

Don’t trust green.
Break it. Then ask Postgres.

BREAK IT ON PURPOSE

Delete one rule from the design, or break the code. **The check must go red.** If it still passes, the check was empty.

A JUDGE YOU CAN’T ARGUE WITH

Same SQL on **real Postgres** and on ours. The diff wins every argument, with the agent and with us.

Postgres is our target, and our judge.

IF YOU REMEMBER ONE THING

You can offload the work.
You can't offload the accountability.

THANK YOU

Questions?

PostgreSQL-compatible TiDB Cloud Starter · Public Preview

Start for free →

tidbcloud.com/free-trial

Ian Zhai · TiDB / PingCAP



Scan to start free